IR Project Document

Task 2 – Text Classification Battle

Name: K R Vaishak
Reg No: RA2411026050018
Class: CSE AIML A
Year: 3rd Year

GitHub Repository

https://github.com/Vaishakkr/IR-Text-Classification-Battle

Project Overview

This project compares four text classifiers – Decision Tree, K-Nearest Neighbours (KNN), Support Vector Machine (SVM) and Naive Bayes – on the BBC News dataset, which has 2,225 real news articles in five classes: business, entertainment, politics, sport and tech. After removing 99 duplicate articles, 2,126 documents were used. Every article is preprocessed with lowercasing, tokenization, stop word removal and Porter stemming, then converted into TF-IDF features. All four models use the same features and the same stratified 80/20 train-test split, and are evaluated with Accuracy, Precision, Recall, F1-score and confusion matrices. Two what-if experiments were also run: changing the TF-IDF vocabulary size and changing the value of k in KNN.

Code and Execution Screenshots

main.py — IR-Text-Classification-Battle

```python
"""
Text Classification Battle - Decision Tree vs KNN vs SVM vs Naive Bayes
Information Retrieval (21CSE377T) - CLA-2, Task 2
Author : K R Vaishak (RA2411026050018), III CSE AIML A

Dataset : BBC News (Greene & Cunningham, ICML 2006) - 2,225 articles, 5 classes
Run     : python main.py
Outputs : results/*.csv, results/*.txt and figures/*.png
"""

import os
import re
import time
import json

import numpy as np
import pandas as pd
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
)

# ---------------------------------------------------------------------
# Configuration (kept in one place so every model sees the same setup)
# ---------------------------------------------------------------------
SEED = 42
TEST_SIZE = 0.20
DATA_FILE = os.path.join("data", "bbc-text.csv")
RESULTS_DIR = "results"
FIGURES_DIR = "figures"

os.makedirs(RESULTS_DIR, exist_ok=True)
os.makedirs(FIGURES_DIR, exist_ok=True)

stemmer = PorterStemmer()
STOP_WORDS = set(ENGLISH_STOP_WORDS)  # built into scikit-learn, no download needed
```

Figure 1: Imports and configuration

main.py — IR-Text-Classification-Battle

```python
# Step 1 : Preprocessing
# ----------------------------------------------------------------------
def preprocess(text):
    """lowercase -> keep alphabetic tokens -> drop stop words / 1-2 letter
    tokens -> Porter stemming. Returns a single space-joined string."""
    text = text.lower()
    tokens = re.findall(r"[a-z]+", text)
    kept = [stemmer.stem(t) for t in tokens if t not in STOP_WORDS and len(t) > 2]
    return " ".join(kept)


def build_vectorizer(max_features=None):
    """The ONE feature representation shared by all four classifiers."""
    return TfidfVectorizer(
        min_df=2,          # ignore terms that appear in only one training document
        max_df=0.95,       # ignore terms present in >95% of documents
        sublinear_tf=True, # tf -> 1 + log(tf), dampens very frequent words
        max_features=max_features,
    )


def build_models():
    """Four classifiers. Hyper-parameters are standard defaults so that the
    comparison is about the algorithm, not about tuning effort."""
    return {
        "Decision Tree": DecisionTreeClassifier(random_state=SEED),
        "KNN (k=5)": KNeighborsClassifier(n_neighbors=5, metric="cosine"),
        "SVM (Linear)": LinearSVC(C=1.0, random_state=SEED),
        "Naive Bayes": MultinomialNB(alpha=1.0),
    }


def evaluate(model, X_tr, y_tr, X_te, y_te):
    t0 = time.perf_counter()
    model.fit(X_tr, y_tr)
    train_time = time.perf_counter() - t0

    t0 = time.perf_counter()
    y_pred = model.predict(X_te)
    pred_time = time.perf_counter() - t0

    p, r, f, _ = precision_recall_fscore_support(y_te, y_pred, average="macro", zero_division=0)
    return y_pred, {
        "Train accuracy": accuracy_score(y_tr, model.predict(X_tr)),
        "Accuracy": accuracy_score(y_te, y_pred),
        "Precision (macro)": p,
        "Recall (macro)": r,
        "F1 (macro)": f,
        "Train time (s)": train_time,
        "Predict time (s)": pred_time,
    }
```

Figure 2: Preprocessing, TF-IDF vectorizer, models and evaluation function

main.py — IR-Text-Classification-Battle

```python
def main():
    log = []

    def say(msg=""):
        print(msg)
        log.append(str(msg))

    say("=" * 70)
    say("TEXT CLASSIFICATION BATTLE : DT vs KNN vs SVM vs NB  (BBC News)")
    say("=" * 70)

    # ------------------------------------------------------------
    # Step 0 : Load and clean dataset
    # ------------------------------------------------------------
    df = pd.read_csv(DATA_FILE)
    say(f"Raw documents loaded          : {len(df)}")
    dup = df.duplicated(subset="text").sum()
    df = df.drop_duplicates(subset="text").reset_index(drop=True)
    say(f"Exact duplicate articles removed : {dup}")
    say(f"Documents used                : {len(df)}")

    dist = df["category"].value_counts().rename_axis("Class").reset_index(name="Documents")
    dist["Percent"] = (100 * dist["Documents"] / len(df)).round(2)
    dist.to_csv(os.path.join(RESULTS_DIR, "class_distribution.csv"), index=False)
    imbalance = dist["Documents"].max() / dist["Documents"].min()
    say("\nClass distribution:")
    say(dist.to_string(index=False))
    say(f"Imbalance ratio (largest / smallest) = {imbalance:.2f}")

    df["raw_words"] = df["text"].str.split().str.len()

    # ------------------------------------------------------------
    # Step 1 : Preprocess
    # ------------------------------------------------------------
    df["clean"] = df["text"].apply(preprocess)
    df["clean_words"] = df["clean"].str.split().str.len()
    say(f"\nAvg words / doc before preprocessing : {df['raw_words'].mean():.1f}")
    say(f"Avg tokens / doc after preprocessing : {df['clean_words'].mean():.1f}")
    sample = df.iloc[0]
    with open(os.path.join(RESULTS_DIR, "preprocessing_example.txt"), "w") as fh:
        fh.write(f"CLASS : {sample['category']}\n\nBEFORE (first 60 words):\n")
        fh.write(" ".join(sample["text"].split()[:60]) + "\n\nAFTER (first 40 tokens):\n")
        fh.write(" ".join(sample["clean"].split()[:40]) + "\n")

    # ------------------------------------------------------------
    # Step 2 : Same stratified split for every model
    # ------------------------------------------------------------
    X_train_txt, X_test_txt, y_train, y_test = train_test_split(
        df["clean"], df["category"], test_size=TEST_SIZE, stratify=df["category"], random_state=SEED
    )
    say(f"\nTrain docs: {len(X_train_txt)}  |  Test docs: {len(X_test_txt)}  (stratified 80/20, seed={SEED})")

    # ------------------------------------------------------------
    # Step 3 : TF-IDF (fit on TRAIN only -> no test leakage)
    # ------------------------------------------------------------
    vec = build_vectorizer()
    X_train = vec.fit_transform(X_train_txt)
    X_test = vec.transform(X_test_txt)
    n_feat = X_train.shape[1]
    sparsity = 100 * (1 - X_train.nnz / (X_train.shape[0] * X_train.shape[1]))
    say(f"TF-IDF vocabulary size: {n_feat}  |  matrix sparsity: {sparsity:.2f}%")

    # ------------------------------------------------------------
    # Step 4 : Train + evaluate the four classifiers
    # ------------------------------------------------------------
    labels = sorted(df["category"].unique())
    rows, preds = [], {}
    say("\n" + "-" * 70)
    for name, model in build_models().items():
        y_pred, m = evaluate(model, X_train, y_train, X_test, y_test)
        preds[name] = y_pred
        rows.append({"Model": name, **m})
        say(f"\n### {name}")
        say(classification_report(y_test, y_pred, digits=4, zero_division=0))

    results = pd.DataFrame(rows)
    results.to_csv(os.path.join(RESULTS_DIR, "model_comparison.csv"), index=False)
    say("-" * 70)
    say("COMPARISON (test set, macro averages)")
    say(results.round(4).to_string(index=False))

    # per-class F1 table
    per_class = {}
    for name, y_pred in preds.items():
        _, _, f, _ = precision_recall_fscore_support(y_test, y_pred, labels=labels, zero_division=0)
        per_class[name] = f
    per_class = pd.DataFrame(per_class, index=labels)
    per_class.to_csv(os.path.join(RESULTS_DIR, "per_class_f1.csv"))
    say("\nPer-class F1:")
    say(per_class.round(4).to_string())
```

Figure 3: Loading data, preprocessing, train-test split, TF-IDF and model training

main.py — IR-Text-Classification-Battle

```python
    # Step 8 : WHAT-IF EXPERIMENTS
    # ------------------------------------------------------------
    # (A) main what-if : shrink the TF-IDF vocabulary for ALL four models
    say("\n" + "=" * 70)
    say("WHAT-IF (A): How does vocabulary size (max_features) affect each model?")
    sizes = [50, 100, 250, 500, 1000, 2500, 5000, None]
    wi_rows = []
    for mf in sizes:
        v = build_vectorizer(max_features=mf)
        Xtr, Xte = v.fit_transform(X_train_txt), v.transform(X_test_txt)
        for name, model in build_models().items():
            _, m = evaluate(model, Xtr, y_train, Xte, y_test)
            wi_rows.append({"max_features": Xtr.shape[1] if mf is None else mf,
                            "Model": name, "Accuracy": m["Accuracy"], "F1 (macro)": m["F1 (macro)"]})
    wi = pd.DataFrame(wi_rows)
    wi.to_csv(os.path.join(RESULTS_DIR, "whatif_vocab_size.csv"), index=False)
    say(wi.pivot(index="max_features", columns="Model", values="F1 (macro)").round(4).to_string())

    fig, ax = plt.subplots(figsize=(9, 5))
    for (name, g), c in zip(wi.groupby("Model", sort=False), colors):
        ax.plot(g["max_features"], g["F1 (macro)"], marker="o", label=name, color=c)
    ax.set_xscale("log")
    ax.set_xlabel("TF-IDF vocabulary size (max_features, log scale)")
    ax.set_ylabel("Macro F1 (test)")
    ax.set_title("What-if A: effect of vocabulary size")
    ax.grid(alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(FIGURES_DIR, "04_whatif_vocab_size.png"), dpi=150)
    plt.close(fig)

    # (B) supporting what-if : KNN neighbours and distance metric
    say("\nWHAT-IF (B): KNN - number of neighbours k and distance metric")
    k_rows = []
    for metric in ["cosine", "euclidean"]:
        for k in [1, 3, 5, 7, 9, 15, 25, 51]:
            _, m = evaluate(KNeighborsClassifier(n_neighbors=k, metric=metric), X_train, y_train, X_test, y_test)
            k_rows.append({"metric": metric, "k": k, "Accuracy": m["Accuracy"], "F1 (macro)": m["F1 (macro)"]})
    kdf = pd.DataFrame(k_rows)
    kdf.to_csv(os.path.join(RESULTS_DIR, "whatif_knn_k.csv"), index=False)
    say(kdf.pivot(index="k", columns="metric", values="F1 (macro)").round(4).to_string())

    fig, ax = plt.subplots(figsize=(8, 4.5))
    for (metric, g), c in zip(kdf.groupby("metric"), ["#55A868", "#8172B2"]):
```

Figure 4: What-if experiments

Output — python3 main.py

```
======================================================================
TEXT CLASSIFICATION BATTLE : DT vs KNN vs SVM vs NB  (BBC News)
======================================================================
Raw documents loaded          : 2225
Exact duplicate articles removed : 99
Documents used                : 2126

Class distribution:
        Class  Documents  Percent
        sport        504    23.71
     business        503    23.66
     politics        403    18.96
entertainment        369    17.36
         tech        347    16.32
Imbalance ratio (largest / smallest) = 1.45

Avg words / doc before preprocessing : 390.5
Avg tokens / doc after preprocessing : 194.6

Train docs: 1700  |  Test docs: 426  (stratified 80/20, seed=42)
TF-IDF vocabulary size: 9881  |  matrix sparsity: 98.74%

----------------------------------------------------------------------
```

Figure 5: Execution output – dataset and preprocessing details

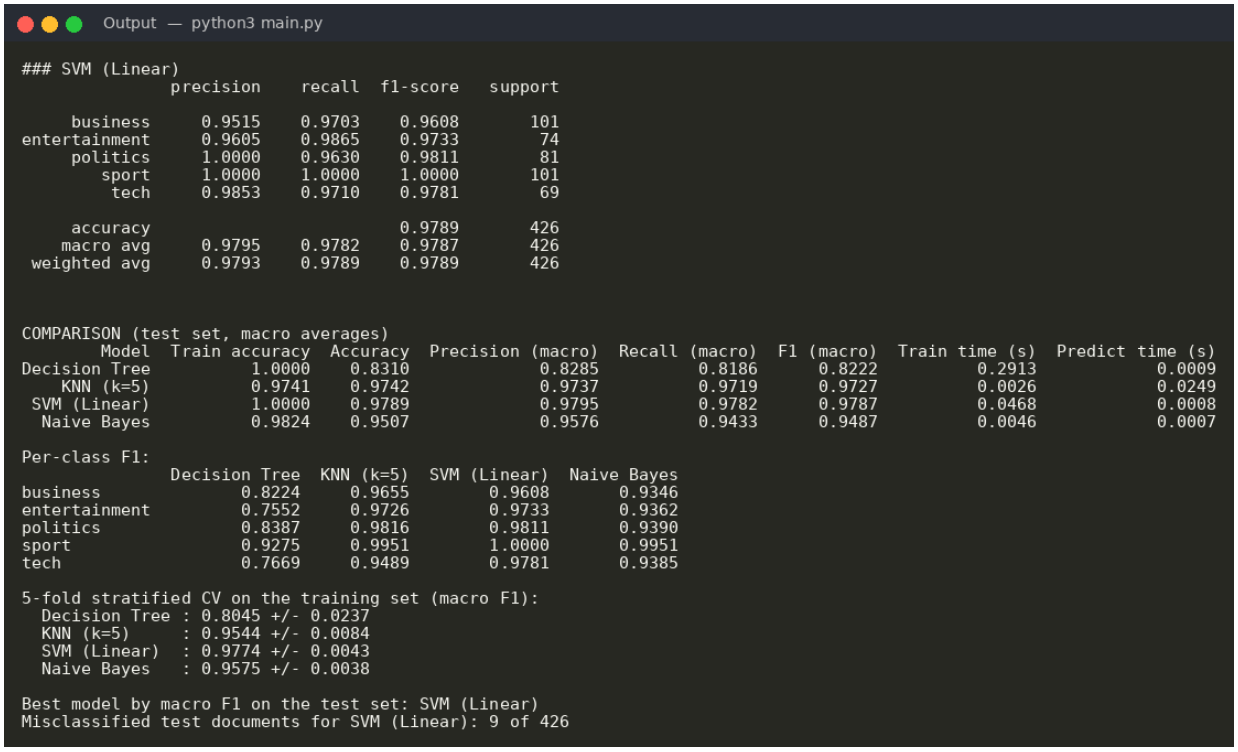


Figure 6: Execution output – SVM report, model comparison and cross-validation

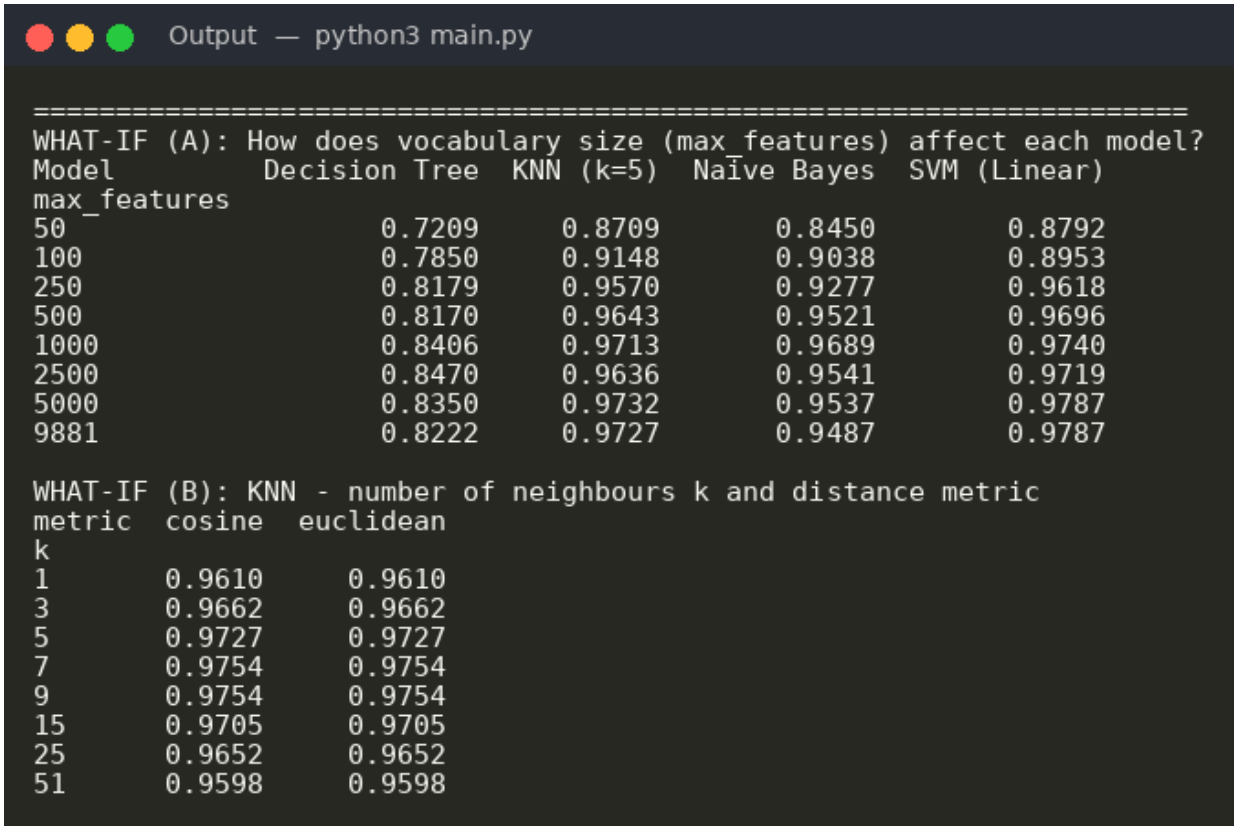


Figure 7: Execution output – what-if experiment results

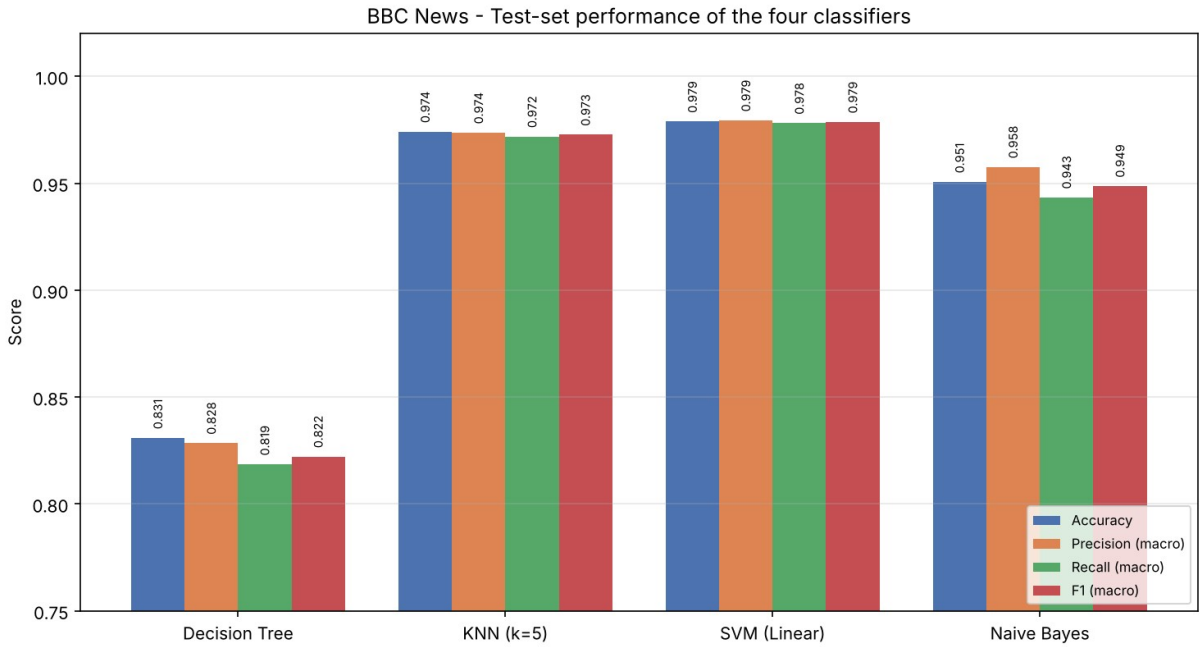


Figure 8: Performance comparison of the four classifiers

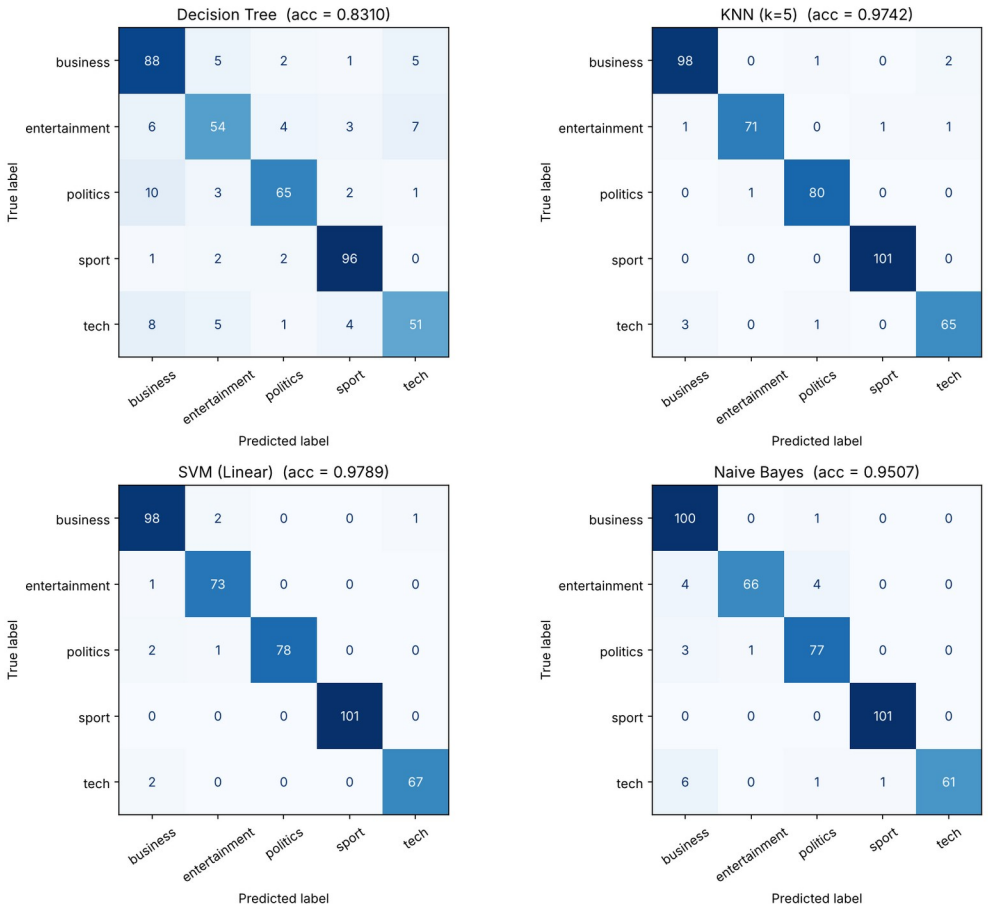


Figure 9: Confusion matrices of the four classifiers

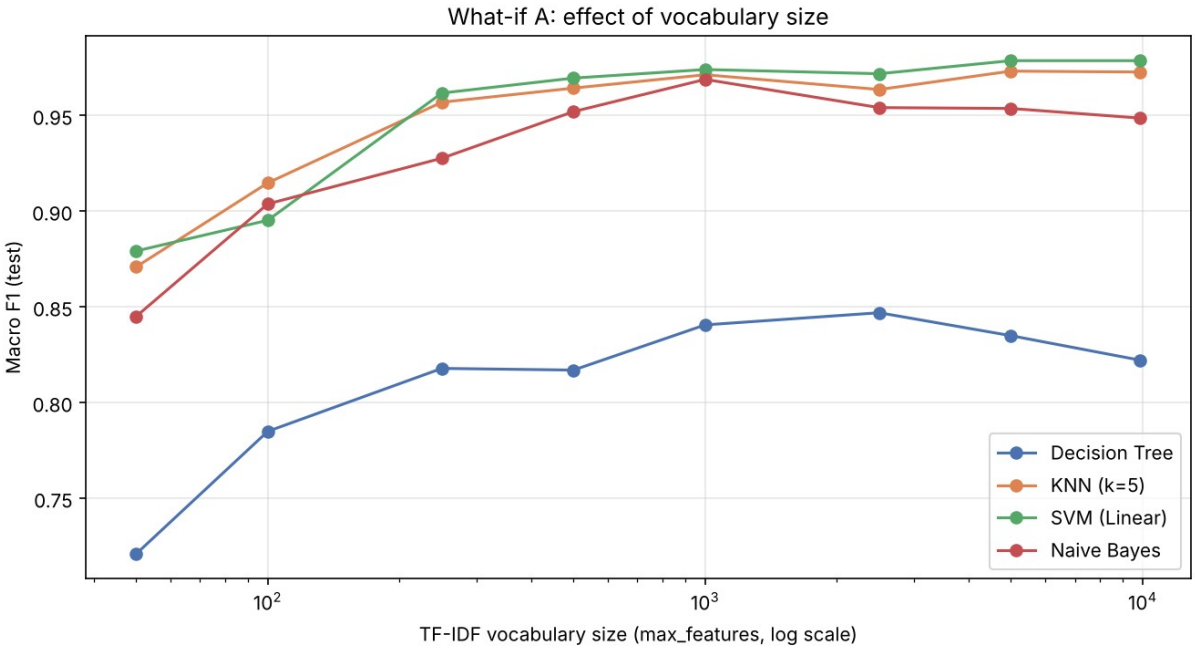


Figure 10: What-if experiment – effect of vocabulary size

Project Result

The execution successfully loaded 2,126 news articles and completed preprocessing, TF-IDF feature extraction with 9,881 terms, and training of all four classifiers. Linear SVM was the best-performing model, with an accuracy of 0.9789 and a macro F1-score of 0.9787, correctly classifying 417 of the 426 test articles. It was followed by KNN (F1 0.9727), Naive Bayes (F1 0.9487) and Decision Tree (F1 0.8222). The Decision Tree overfit, scoring 100% on training data but 83.1% on test data. In the what-if experiments, Naive Bayes performed best with 1,000 features (F1 0.9689), and KNN performed best at k = 7 to 9 (F1 0.9754). Based on these results, Linear SVM is recommended for this news classification task.